ETSI
ESCUELA TÉCNICA SUPERIOR DE INGENIERÍA

# PRÁCTICA #2

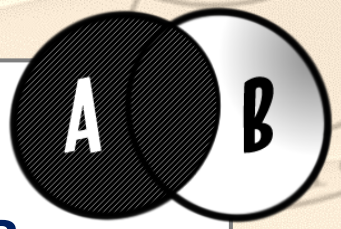
▷ Ejercicios-Sesión#5

# TIPOS DE JOIN (Recordatorio)

## a. INNER JOIN

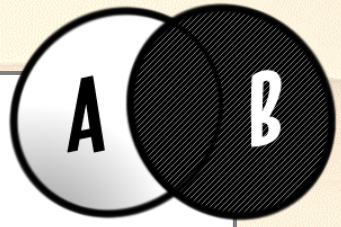
```
SELECT <lista_campos>
FROM Tabla_A  A
        INNER JOIN  Tabla_B  B
        ON A.Clave (PK) = B.Clave (FK);
```

## b. LEFT JOIN



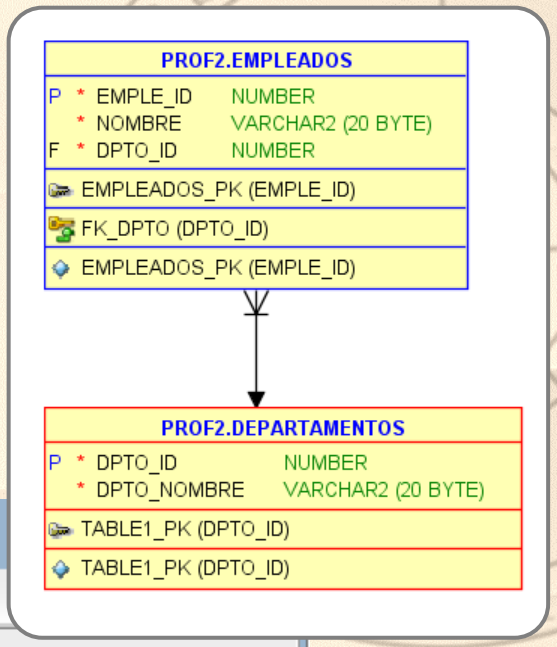

```
SELECT <lista_campos>
FROM Tabla_A  A
        LEFT JOIN  Tabla_B  B
        ON A.Clave (PK) = B.Clave (FK);
```

## c. RIGHT JOIN




```
SELECT <lista_campos>
FROM Tabla_A  A
        RIGHT JOIN  Tabla_B  B
        ON A.Clave (PK) = B.Clave (FK);
```

# a. INNER JOIN




Rabida | DEPARTAMENTOS | 0,207 segundos | Rabida

Hoja de Trabajo | Generador de Consultas

```
1  select * from empleados;
2  select * from departamentos;
```

Salida de Script | Resultado de la Consulta

Tarea terminada en 0,207 segundos

```
DPTO_ID     NOT NULL NUMBER
DPTO_NOMBRE NOT NULL VARCHAR2(20)

  EMPLE_ID NOMBRE                    DPTO_ID
---------- -------------------- ----------
         1 MORTADELO                     1
         2 FILEMÓN                       2
         3 PEPE GOTERA                   1
         4 OTILIO

   DPTO_ID DPTO_NOMBRE
---------- --------------------
         1 Informática
         2 Contabilidad
         3 RRHH
```

NULO

Rabida | DEPARTAMENTOS

Hoja de Trabajo | Generador de Consultas

```
1  select * from empleados A
2                inner join departamentos B
3                on A.dpto_id = B.dpto_id;
4
```

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 3 en 0,027 segundos

| | EMPLE_ID | NOMBRE | DPTO_ID | DPTO_ID_1 | DPTO_NOMBRE |
|---|---|---|---|---|---|
| 1 | 1 | MORTADELO | 1 | 1 | Informática |
| 2 | 3 | PEPE GOTERA | 1 | 1 | Informática |
| 3 | 2 | FILEMÓN | 2 | 2 | Contabilidad |

OTILIO

*Sólo registros de Empleados que tienen asociados un Departamento*

# b. LEFT JOIN

Rabida | DEPARTAMENTOS | 0,207 segundos | Rabida

Hoja de Trabajo | Generador de Consultas

```
1  select * from empleados;
2  select * from departamentos;
```

Salida de Script | Resultado de la Consulta | Tarea terminada en 0,207 segundos

```
DPTO_ID     NOT NULL NUMBER
DPTO_NOMBRE NOT NULL VARCHAR2(20)

  EMPLE_ID NOMBRE                    DPTO_ID
---------- -------------------- ----------
         1 MORTADELO                     1
         2 FILEMÓN                       2
         3 PEPE GOTERA                   1
         4 OTILIO

   DPTO_ID DPTO_NOMBRE
---------- --------------------
         1 Informática
         2 Contabilidad
         3 RRHH
```

NULO

Rabida | Rabida

Hoja de Trabajo | Generador de Consultas

```
1  select * from empleados A
2                left join departamentos B
3                on A.dpto_id = B.dpto_id;
4
```

Salida de Script | Resultado de la Consulta | SQL | Todas las Filas Recuperadas: 4 en 0,028 segundos

| | EMPLE_ID | NOMBRE | DPTO_ID | DPTO_ID_1 | DPTO_NOMBRE |
|---|---|---|---|---|---|
| 1 | 3 | PEPE GOTERA | 1 | 1 | Informática |
| 2 | 1 | MORTADELO | 1 | 1 | Informática |
| 3 | 2 | FILEMÓN | 2 | 2 | Contabilidad |
| 4 | 4 | OTILIO | (null) | (null) | (null) |

OTILIO

*Se muestran TODOS los registros de Empleados tengan o no asociados un Departamento*

# c. RIGHT JOIN

Rabida | DEPARTAMENTOS | 0,207 segundos | Rabida

Hoja de Trabajo | Generador de Consultas

```
1  select * from empleados;
2  select * from departamentos;
```

Salida de Script | Resultado de la Consulta | Tarea terminada en 0,207 segundos

```
DPTO_ID     NOT NULL NUMBER
DPTO_NOMBRE NOT NULL VARCHAR2(20)

  EMPLE_ID NOMBRE                    DPTO_ID
---------- -------------------- ----------
         1 MORTADELO                     1
         2 FILEMÓN                       2
         3 PEPE GOTERA                   1
         4 OTILIO

   DPTO_ID DPTO_NOMBRE
---------- --------------------
         1 Informática
         2 Contabilidad
         3 RRHH
```

NO TIENE EMPLEADOS ASOCIADOS

Rabida | Rabida

Hoja de Trabajo | Generador de Consultas

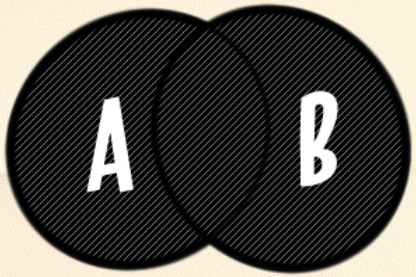
```
1  select * from empleados A
2                 right join departamentos B
3                 on A.dpto_id = B.dpto_id;
4
```

Salida de Script | Resultado de la Consulta | SQL | Todas las Filas Recuperadas: 4 en 0,032 segundos

| | EMPLE_ID | NOMBRE | DPTO_ID | DPTO_ID_1 | DPTO_NOMBRE |
|---|---|---|---|---|---|
| 1 | 1 | MORTADELO | 1 | 1 | Informática |
| 2 | 3 | PEPE GOTERA | 1 | 1 | Informática |
| 3 | 2 | FILEMÓN | 2 | 2 | Contabilidad |
| 4 | (null) | (null) | (null) | 3 | RRHH |

*Se muestran TODOS los registros de Departamentos tengan o no asociados Empleados*

# d. FULL/FULL OUTER JOIN




```
SELECT <lista_campos>
FROM Tabla_A  A
        FULL OUTER JOIN  Tabla_B  B
        ON A.Clave (PK) = B.Clave (FK);
```

Rabida | DEPARTAMENTOS | 0,207 segundos

Hoja de Trabajo | Generador de Consultas

```
1  select * from empleados;
2  select * from departamentos;
```

Salida de Script | Resultado de la Consulta

Tarea terminada en 0,207 segundos

```
DPTO_ID     NOT NULL NUMBER
DPTO_NOMBRE NOT NULL VARCHAR2(20)

  EMPLE_ID NOMBRE                    DPTO_ID
---------- -------------------- ----------
         1 MORTADELO                     1
         2 FILEMÓN                       2
         3 PEPE GOTERA                   1
         4 OTILIO

   DPTO_ID DPTO_NOMBRE
---------- --------------------
         1 Informática
         2 Contabilidad
         3 RRHH
```

NULO

NO TIENE EMPLEADOS ASOCIADOS

Rabida

Hoja de Trabajo | Generador de Consultas

```
1  select * from empleados A
2                 full outer join departamentos B
3                 on A.dpto_id = B.dpto_id;
4
```

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 5 en 0,048 segundos

| | EMPLE_ID | NOMBRE | DPTO_ID | DPTO_ID_1 | DPTO_NOMBRE |
|---|---|---|---|---|---|
| 1 | 1 | MORTADELO | 1 | 1 | Informática |
| 2 | 2 | FILEMÓN | 2 | 2 | Contabilidad |
| 3 | 3 | PEPE GOTERA | 1 | 1 | Informática |
| 4 | 4 | OTILIO | (null) | (null) | (null) |
| 5 | (null) | (null) | (null) | 3 | RRHH |

*Se muestran TODOS los registros de ambas tablas, tengan o no registros asociados*

# 1. FUNCIONES EN ORACLE SQL

- Las **funciones** se usan dentro de expresiones y actúan sobre los valores de las *columnas*, *variables* o *constantes*.
- Generalmente producen *dos tipos diferentes de resultados*:
  - Unas ***producen una modificación de la información original*** (por ejemplo, *convertir a minúsculas una cadena que esté en mayúsculas*);
  - otras ***indican algún dato sobre la información*** que reciba como parámetro (por ejemplo, *devolver el número de caracteres que tiene una cadena*).
- Las **funciones** se pueden utilizar en cláusulas SELECT, cláusulas WHERE y cláusulas ORDER BY.
- Es posible realizar el **anidamiento** de funciones.
- Existen cinco tipos de funciones principales, en función del *tipo de datos* que manejen: ***aritméticas***, *de cadenas de caracteres, de manejo de fechas, de conversión* y *otras funciones*.

# 1.1 FUNCIONES ARITMÉTICAS ORACLE SQL

- Las **funciones aritméticas** trabajan con *datos de tipo numérico* NUMBER. Este tipo incluye los ***dígitos***, de 0 a 9, el ***punto decimal*** y el ***signo menos***, si es necesario.
- *Los valores numéricos no se encierran entre comillas*.
- Este tipo de funciones pueden trabajar con **tres clases de números**:
  - *Valores simples*
  - *Grupos de valores*
  - *Listas de valores*
- Algunas **funciones** ***modifican*** los valores sobre los que actúan; otras ***informan*** de algún dato descriptivo de los valores.
  - Funciones de *Valores simples*
  - Funciones de *Grupos de valores*
  - Funciones de *Listas de valores*

# 1.1.1 FUNCIONES ARITMÉTICAS DE VALORES SIMPLES

- Las ***funciones aritméticas de valores simples*** son funciones sencillas que trabajan con *valores numéricos simples*.
- Algunas de las funciones de este tipo serían las siguientes
  - **ABS(**n**):** Devuelve el ***valor absoluto*** de '**n**'. El valor absoluto es siempre un número *positivo*.
  - **CEIL(**n**):** Obtiene el *valor entero inmediatamente superior o igual a* '**n**'.
  - **FLOOR(**n**):** Es lo opuesto a CEIL, devuelve el *valor entero inmediatamente inferior o igual a* '**n**'.
  - **MOD(**m, n**):** Devuelve el *resto* resultante de *dividir* 'm' *entre* 'n'.
  - **NVL(***valor*, *expresión***):** Esta función se utiliza para sustituir un *valor nulo* por otro valor. Si *'valor'* es NULL, entonces es sustituido por la *'expresión'*; si no lo fuese, la función devuelve *'valor'*. NVL se puede usar con cualquier tipo de datos: numéricos, carácter, tipo fecha, pero *'valor'* y *'expresión'* deben ser del mismo tipo.

EJs.

- **ABS(n). Obtén el valor absoluto del SALARIO-10000 para todas las filas de la tabla EMPLE:**

```
SQL> SELECT APELLIDO,SALARIO, ABS(SALARIO-10000) FROM EMPLE;
```

- **CEIL(n). Prueba con números positivos y negativos:**

```
SQL> SELECT CEIL(20.3), CEIL(16), CEIL(-20.3), CEIL(-16) FROM DUAL;

CEIL(20.3)   CEIL(16)   CEIL(-20.3)   CEIL(-16)
-----------  ----------  -------------  ----------
        21          16            -20         -16
```

- **FLOOR(n). Prueba con números positivos y negativos:**

```
SQL> SELECT FLOOR(20.3), FLOOR(16), FLOOR(-20.3), FLOOR(-16)
     FROM DUAL;

FLOOR(20.3)   FLOOR(16)   FLOOR(-20.3)   FLOOR(-16)
---------------  -------------  ----------------  -------------
         20            16              -21           -16
```

EJs.

**SQRT(valor): Prueba con números positivos y negativos:**

```
SQL> SELECT SQRT(25), SQRT(25.6) FROM DUAL;

SQRT(25)     SQRT(25.6)
-----------  --------------
        -5   5,05964426
```

**NVL(valor, expresión). A partir de la tabla EMPLE obtenemos el SALARIO, la COMISION y la suma de ambos:**

```
SQL> SELECT SALARIO,COMISION, SALARIO+COMISION FROM EMPLE;

SALARIO   COMISION     SALARIO+COMISION
----------  -------------  ------------------------
      1040                                  NULL
      1500           390                    1890
      1625           650                    2275
      2900
      1600          1020                    2620
      3005
```

Si la *comisión* es nula, la columna SALARIO+COMISION da como resultado un valor *nulo* (no se visualiza nada). Aplicando la función NVL a la *comisión*, de manera que si es nula, sustituimos su valor por 0, veremos que se obtienen valores para todas las tuplas en dicha columna

EJs.

**NVL(**valor, expresión**). A partir de la tabla EMPLE obtenemos el SALARIO, la COMISION y la suma de ambos:**

```
SQL> SELECT SALARIO,COMISION, SALARIO+NVL(COMISION,0)
     FROM EMPLE;
```

| SALARIO | COMISION | SALARIO+COMISION |
|---|---|---|
| 1040 | | 1040 |
| 1500 | 390 | 1890 |
| 1625 | 650 | 2275 |
| 2900 | | 2900 |
| 1600 | 1020 | 2620 |
| 3005 | | 3005 |

COMISION=0

NULL

Vemos que ahora sí se obtienen valores para todas las tuplas en la columna SALARIO+COMISION

**POWER(**m, exponente**). Prueba con números positivos y negativos:**

```
SQL>SELECT POWER(2,4),POWER(2,-4),POWER(3.5,2.4),POWER(4.5,2)
     FROM DUAL;
```

| POWER(2,4) | POWER(2,-4) | POWER(3.5,2.4) | POWER(4.5,2) |
|---|---|---|---|
| 16 | ,0625 | 20,2191692 | 20,25 |

# 1.1.2 FUNCIONES ARITMÉTICAS DE GRUPOS DE VALORES

- Además de las *funciones aritméticas de valores simples* existen otras ***funciones estadísticas***, como SUM, AVG o COUNT, que ***actúan sobre un grupo de filas para obtener de él un valor descriptivo*** *(suma de valores, valor máximo, etc)*.
- Estas funciones permiten obtener, por ejemplo, la *edad media* de un grupo de alumnos, el alumno *más joven*, el *más viejo* o el *número total de miembros* de un grupo.
- **Los valores *nulos son ignorados* por las *funciones de grupos de valores* y los cálculos se realizan ⚠*sin contar con ellos*.**
- En todas las ***funciones de grupo***, al indicar los argumentos *se pueden emplear las cláusulas* **DISTINCT** (se *obtienen las filas resultantes* ***eliminando duplicados***) y **ALL** (***recoge todas las filas*** *aunque sus valores estén duplicados*), aunque dichas cláusulas no se suelen utilizar en las funciones AVG, SUM, MAX ni MIN, pero sí es más normal su uso en COUNT.

- **AVG(**n**):** Calcula el ***valor medio*** de '**n**' ⚠ ***ignorando los valores nulos***.

  EJ: ***Calcular el salario medio de los empleados, de la tabla EMPLE, pertenecientes al departamento 10 :***

```
SQL> SELECT AVG(SALARIO) FROM EMPLE WHERE DEPT_NO=10;

     AVG(SALARIO)
     ----------------
       2891,66667
```

- **COUNT (*|***expresión***)**

  ⚠ La opción '*' cuenta el **número de elementos en un conjunto**, ⚠ ***incluyendo valores NULL y duplicados***.

  ☹ Si COUNT recibe como argumento una expresión o columna, ésta podrá ir precedida de las cláusulas ALL o DISTINCT, y el formato sería el siguiente:

  **COUNT ( * | [**DISTINCT **|** ALL**]** *expresión***)**

  - **COUNT(ALL** expresión**):** evalúa la expresión para cada fila de un conjunto y devuelve el ***número de valores <u>no nulos</u>***.
  - **COUNT(DISTINCT** expresión**):** evalúa la expresión para cada fila de un conjunto y devuelve el ***número de valores <u>únicos</u> <u>no nulos</u>***.

**EJ.** ***Cálculo del número de filas de la tabla EMPLE:***

```
SQL> SELECT COUNT(*) FROM EMPLE;

COUNT(*)
----------
        14
```

Esta consulta contabiliza todos los registros de la tabla EMPLE, *incluyendo valores nulos y repetidos*

**EJ.** ***Cálculo del número de filas de la tabla EMPLE:***

```
SQL> SELECT COUNT(ALL COMISION) FROM EMPLE;

COUNT(*)
----------
         4
```

Esta consulta contabiliza los valores del campo COMISIÓN que *no son nulos*

**EJ.** ***Cálculo del número de oficios que hay en la tabla EMPLE:***

```
SQL> SELECT COUNT(DISTINCT OFICIO) 'OFICIOS' FROM EMPLE;

OFICIOS
---------
        5
```

Esta consulta cuenta todos los oficios de la tabla EMPLE que *no sean nulos, eliminando repeticiones*

- **MAX(*expresión*):** Calcula el valor máximo de '*expresión*'.

**EJ.** ***Cálculo del salario máximo de la tabla EMPLE:***

```
SQL> SELECT MAX(SALARIO) FROM EMPLE;
     MAX(SALARIO)
     ------------
            4100
```

**EJ.** ***Obtener el apellido del empleado que tiene el mayor salario:***

```
SQL> SELECT APELLIDO, SALARIO FROM EMPLE
     WHERE SALARIO=(SELECT MAX(SALARIO) FROM EMPLE);

     APELLIDO   SALARIO
     ---------- --------
          REY      4100
```

⚠ **Se necesita una subconsulta para calcular el máximo salario y compararlo después con el salario de cada uno de los empleados de la tabla EMPLE**

- **MIN(*expresión*):** Calcula el valor mínimo de '*expresión*'.

EJ. ***Cálculo del mínimo salario de la tabla EMPLE:***

```
SQL> SELECT MIN(SALARIO) FROM EMPLE;
     MIN(SALARIO)
     ------------
            1040
```

EJ. ***Obtén el apellido del empleado que tiene el menor salario:***

```
SQL> SELECT APELLIDO, SALARIO FROM EMPLE
     WHERE SALARIO=(SELECT MIN(SALARIO) FROM EMPLE);

     APELLIDO   SALARIO
     ---------- --------
      SANCHEZ      1040
```

**En primer lugar, se selecciona el salario mínimo y, a continuación, se compara con los salarios de la tabla EMPLE para obtener el apellido que tiene ese salario mínimo**

- **SUM(*expresión*):** Obtiene la *suma* de valores de *'expresión'* ⚠ ***distintos de nulos***

**EJ.** ↳ ***Cálculo de la suma de todos los salarios de la tabla EMPLE:***

```
SQL> SELECT SUM(SALARIO) FROM EMPLE;

     SUM(SALARIO)
     ------------
           30460
```

- **VARIANCE(*expresión*):** Obtiene la *varianza* de los valores de *'expresión'* ⚠ ***distintos de nulos***

**EJ.** ↳ ***Obtener la varianza de todos los salarios de la tabla EMPLE:***

```
SQL> SELECT VARIANCE(SALARIO) FROM EMPLE;

     VARIANCE(SALARIO)
     -----------------
           72226,374
```

# 1.2 FUNCIONES DE CADENAS ORACLE SQL - I

| Función | Propósito |
|---|---|
| **CHR(**n**)** | Devuelve el carácter cuyo valor en binario es equivalente a '**n**'. |
| **CONCAT(***cad1*, *cad2***)** | Devuelve *'cad1' concatenada* con *'cad2'*. Es equivalente al operador \|\|. |
| **LOWER(***cad***)** | Devuelve la cadena *'cad' con todas sus letras convertidas a minúsculas*. |
| **UPPER(***cad***)** | Devuelve la cadena *'cad' con todas sus letras convertidas a mayúsculas*. |
| **INITCAP(***cad***)** | Convierte la cadena *'cad'* a tipo *título*, la primera letra de cada palabra de *'cad'* a mayúsculas y el resto, a minúsculas. **Ej. INITCAP(**'oRACle Y sqL**')** ➜ **'Oracle Y Sql'** |
| **LPAD(***cad1*, n [, *cad2*]**)** | Añade caracteres a la izquierda de *'cad1'* hasta que alcance la longitud '*n*'. Devuelve *'cad1'* con longitud '*n*' y *ajustado a la derecha*; *'cad2'* es la cadena con la que se rellena por la izquierda; cad1 puede ser una columna de una tabla o cualquier literal. Si *'cad2'* se suprime, asume como carácter de *relleno el blanco*.<br>**Ej.** *obtenemos en una columna el sueldo de los empleados con una longitud de 10 caracteres y rellenando por la izquierda con asteriscos:*<br>**SQL> SELECT LPAD(SUELDO,10,'*') FROM EMPLE; ➜ '******2100'** |
| **RPAD(***cad1*, n [, *cad2*]**)** | Añade caracteres a la derecha de *'cad1'* hasta que alcance una cierta longitud '*n*'. Devuelve *'cad1'* con longitud '*n*' y ajustado a la izquierda; *'cad2'* es la cadena con la que se rellena por la derecha; *'cad1'* puede ser una columna de una tabla o cualquier literal. Si *'cad2'* se suprime, asume como carácter de relleno el blanco.<br>**Ej. SQL> SELECT RPAD(SUELDO,10,'*') FROM EMPLE; ➜ '2100******'** |

# 1.2 FUNCIONES DE CADENAS ORACLE SQL-II

| Función | Propósito |
|---|---|
| **LTRIM(*cad* [, set])** | Suprime un conjunto de caracteres a la izquierda de la cadena 'cad'; 'set' es el conjunto de caracteres a suprimir. Devuelve 'cad' con el grupo de caracteres 'set' omitidos por la izquierda. Si el segundo parámetro se omite, devuelve la mismacadena. Por defecto, si la cadena contiene blancos a la izquierda y se omite el segundo parámetro, la función devuelve la cadena sin blancos a la izquierda.<br>**Ej. *Quitamos los caracteres punto y comilla de la derecha de la columna TITULO:***<br>**SQL> SELECT RTRIM (TITULO, ' .” ' ) FROM MISTEXTOS;** |
| **RTRIM(*cad* [, set])** | Suprime un conjunto de caracteres a la derecha de la cadena *'cad'*. Devuelve *'cad'* con el grupo de caracteres *'set'* omitidos por la derecha. Si se omite *'set'*, devuelve *'cad'* tal como está. Por defecto, si la cadena contiene blancos a la derecha y se omite el segundo parámetro, la función devuelve la cadena sin blancos a la derecha. |
| **REPLACE (*cad*, cad_búsqueda [,cad_sustitución])** | Sustituye un carácter o varios caracteres de una cadena con 0 ó más caracteres. Devuelve *'cad'* con cada ocurrencia de *'cad_búsqueda'* sustituida por *'cad_sustitución'*.<br>**SQL> SELECT REPLACE('BLANCO Y NEGRO', 'O', 'AS') FROM DUAL; ➔ BLANCAS Y NEGRAS** |
| **SUBSTR (cad, m [,n])** | Obtiene parte de una cadena. Devuelve la subcadena de *'cad'*, que abarca desde la posición indicada en 'm' hasta tantos caracteres como indique el número '*n*'. Si se omite '*n*', devuelve la cadena desde la posición especificada por '*m*'. El valor de 'n' no puede ser inferior a *1*. El valor de '*m*' puede ser *negativo*; en ese caso, devuelve la cadena empezando por su final, y avanzando de derecha a izquierda.<br>**Ej. SQL> SELECT SUBSTR('ABCDEFG',3,2) "s1", SUBSTR ('ABCDEFG',-3,2) FROM DUAL;**<br>**➔ s1:'CD' y s3:'EF'** |

# 3. AGRUPACIÓN DE ELEMENTOS

- A veces, nos interesa consultar los datos según grupos determinados, para lo cual, *las cláusulas que conocemos hasta ahora no son suficientes*. Necesitamos una *cláusula que permita realizar agrupamientos por campo*: **GROUP BY.**
- El ***agrupamiento*** se lleva a cabo mediante la cláusula **GROUP BY** ***por las columnas especificadas y en el orden especificado***. Su formato es el siguiente:

```
SELECT ...
FROM ...
GROUP BY columna1, columna2, columna3,...
HAVING condición
ORDER BY...
```

- **Los datos seleccionados en la sentencia SELECT que lleva el GROUP BY deben ser: una constante, una función de grupo (**SUM, COUNT, AVG, **...) o una columna expresada en el GROUP BY.** ⚠*No pueden incluir atributos cuyos valores puedan variar para un mismo grupo*.

**EJ.** ***Se desea conocer cuál es el salario medio de cada departamento de la tabla EMPLE.***

```
SELECT DEPT_NO, AVG(SALARIO)
FROM EMPLE
GROUP BY DEPT_NO;
```

- La cláusula **GROUP BY** sirve para calcular ***propiedades de uno o más conjuntos de filas: suma, media, valor máximo,…***.
- Del mismo modo que existe la condición de búsqueda **WHERE** para filas individuales, también hay una condición de búsqueda para grupos de filas: **HAVING**.
- La cláusula **HAVING** se emplea para controlar cuál de los conjuntos de filas se deberán considerar: ***sólo a aquellos grupos de tuplas que satisfagan las condiciones*** *que establezcamos*. Se evalúa sobre la tabla que devuelve el GROUP BY.
- ⚠ La cláusula **HAVING** *no puede existir* sin **GROUP BY**.

**EJ.** ▷ ***Visualizar, a partir de la tabla EMPLE, el número de empleados que hay en cada departamento.***

Para hacer esta consulta, tenemos que *agrupar* las filas de la tabla EMPLE por *departamento* (**GROUP BY** DEPT_NO) y contabilizarlas (**COUNT(*)**). La consulta sería la siguiente:

```
SQL> SELECT DEPT_NO, COUNT(*) FROM EMPLE GROUP BY DEPT_NO;
```

| DEPT_NO | COUNT(*) |
|---|---|
| 10 | 3 |
| 20 | 5 |
| 30 | 6 |

- COUNT es una *función de grupo* y da información sobre un **grupo de filas**, no sobre filas individuales de la tabla.
- La cláusula GROUP BY DEPT_NO obliga a COUNT a contar *las filas que se han agrupado por cada departamento*.

Si en la consulta anterior, sólo quisiéramos visualizar los departamentos *con más de 4 empleados*, tendríamos que escribir lo siguiente:

```
SQL> SELECT DEPT_NO, COUNT(*)
     FROM EMPLE
     GROUP BY DEPT_NO
     HAVING COUNT(*) > 4;
```

| DEPT_NO | COUNT(*) |
|---|---|
| 20 | 5 |
| 30 | 6 |

>4

**EJ.** ***Si queremos ordenar la salida descendentemente por número de empleados, obteniendo, utilizamos la siguiente orden ORDER BY:***

- Cuando usamos la cláusula ORDER BY con columnas y funciones de grupo hemos de tener en cuenta que **ésta se ejecuta detrás de las cláusulas WHERE, GROUP BY y HAVING**.
- **En ORDER BY podemos especificar *funciones de grupo, columnas de GROUP BY* o su *combinación*.**

```
SQL> SELECT DEPT_NO, COUNT(*) FROM EMPLE
     GROUP BY DEPT_NO
     HAVING COUNT(*) > 4
     ORDER BY COUNT(*) DESC;
```

**La evaluación de las cláusulas en tiempo de ejecución se efectúa en el siguiente orden:**

- **WHERE** Selecciona las filas.
- **GROUP BY** Agrupa estas filas.
- **HAVING** Filtra los grupos. Selecciona y elimina los grupos.
- **ORDER BY** Clasifica la salida. Ordena los grupos.

# 4. DIVISIÓN EN SQL

- La operación de **División** se utiliza en consultas en las que se busca que algún *atributo de una relación se relacione con **TODOS** los valores de otro atributo de otra relación* (*cuantificación universal*).
- Recordando la definición del álgebra relacional para esta operación, la **DIVISION**, **R÷S**:
  - Se aplica a dos relaciones, **R**(X) y **S**(Y), de grados *n* y *m*, respectivamente, con **Y** ={$y_1$,..,$y_m$ } ⊆ **X** {$x_1$,..,$x_n$,$y_1$,..,$y_m$} (*Y es subconjunto propio de X*).
  - El resultado de la DIVISIÓN es una relación, **T**(Z), de grado ***p***=*n*, cuyos atributos son ***Z***=*X*−*Y* (*atributos de R no están en S)*, que incluye las tuplas, t=($z_1$,..,$z_p$), tomadas a partir de las tuplas de R(X), tales que ***sus valores*** ($x_1$,..,$x_n$) **en R *están asociados con todos los valores*** ($y_1$,..,$y_m$) ***en S***. Es decir, para que una tupla **t** se añada en **T**, los valores de **t** deben aparecer en **R** combinados con TODAS las tupla de **S**.

## EJ.

R(X,Y) ÷ S(Y) = R(X,Y)÷S(Y)

| X | Y |
|---|---|
| $x_1$ | $y_1$ |
| $x_1$ | $y_2$ |
| $x_2$ | $y_1$ |
| $x_3$ | $y_2$ |

÷

| Y |
|---|
| $y_1$ |
| $y_2$ |

=

| Z(X) |
|---|
| $x_1$ |

ℹ X1 es el único valor de R(X,Y) que se combina con TODOS los valores de S(Y)

**EJ.** Dado el siguiente esquema de relación, obtener el DNI de los *empleados* que trabajan *en todos los proyectos* en los que trabaja el empleado con DNI=123.

*Emple*(**dni**, nom) *Proy*(**npro**,dni)

## SOLUCIÓN:

**1** *Obtenemos los ids (npro) de todos los proyectos en los que trabaja el empleado con dni=123 :*

$$temp \leftarrow \pi_{npro}(\sigma_{dni=123}(Proy))$$

**2** *Obtenemos los dnis de TODOS los empleados que se trabajen en TODOS los proyectos (los dnis de la tabla **Proy** que se combinen con **todos** los valores de la tabla **Temp**)*

$$res \leftarrow Proy \div temp$$

EMPLE

| DNI | NOMBRE |
|---|---|
| 123 | ANACLETO |
| 456 | CARPANTA |
| 789 | OTILIO |

PROY

| DNI | NPRO |
|---|---|
| 123 | 1 |
| 123 | 2 |
| 456 | 1 |
| 456 | 2 |
| 456 | 3 |
| 789 | 1 |

TEMP (1)

| NPRO |
|---|
| 1 |
| 2 |

PROY ÷ TEMP

RES (2)

| DNI |
|---|
| 123 |
| 456 |

- El lenguaje SQL ***no proporciona un operador*** que implemente directamente la ***división***.
- Existen distintas formas de realizar la operación división en SQL. Una de ellas, es la basada en el uso del ***doble NOT EXISTS***, en la que tendríamos que ***replantear la consulta en otros términos***.
- El bloque de doble NOT EXISTS se basa en una *equivalencia lógica*:

$$\forall x: P(x) \leftrightarrow \neg \exists\, x: \neg P(x)$$

  - ***Si para todo x se cumple P(x), entonces, no existe un x para el que no se cumpla P(x). Ej:*** *Si un alumno tiene notas en todas las asignaturas, podremos afirmar que no existe una asignatura en la que no tenga nota dicho alumno.*
- SQL tiene un operador NOT EXISTS, por lo que podremos resolver la consulta si la logramos expresar en términos de dicho operador. Ej. El ejemplo anterior podríamos replantearlo como ***'Buscar los alumnos para los que no exista asignatura en la que no exista nota de ese alumno en dicha asignatura'.***

## ▷ESQUEMA GENERAL:

CORRELACIÓN 1

CORRELACIÓN 2

```
SELECT DISTINCT X.a FROM T1 AS X
 WHERE NOT EXISTS ( SELECT * FROM T2 AS Y
                    WHERE NOT EXISTS ( SELECT * FROM T3 AS Z
                                       WHERE (Z.a=X.a)
                                         AND (Z.b=Y.b)) );
```

**EJ.** Suponiendo una BD con las tablas: *Alumno*, *Cursos_Requeridos* y *Cursos_Completados*, obtener todos los alumnos que pueden graduarse, es decir, aquéllos que han completado todos los cursos requeridos para poder obtener la graduación. La consulta podremos replantearla como: *para todos los alumnos que se van a graduar, no existe ningún curso requerido que no haya completado el alumno*.

```
SELECT DISTINCT X.idAlum FROM Alumno X
  WHERE NOT EXISTS( SELECT * FROM Curso_Requerido AS Y
                    ´WHERE NOT  EXISTS( SELECT *
                                        FROM Curso_Completado AS Z
                                        WHERE Z.Curso=Y.Curso
                                          AND Z.idAlum=X.idAlum ));
```

**EJ.** Suponiendo una BD con las tablas: *Clientes*, que contiene un campo '*nombre*', y *Frecuenta*, que contiene dos campos, *Cliente* y Tiendas (que *frecuenta* el cliente), obtener TODOS los *clientes* que frecuentan TODAS las *tiendas*. La consulta podremos replantearla como: *Seleccionar todos los clientes tales que no haya una tienda que no frecuenten*.

Seleccionamos todos los clientes que frecuenten TODAS las tiendas

CORRELACIÓN 1

```
SELECT CLTE.idClte
FROM Clientes CLTE
WHERE NOT EXISTS (
    SELECT *
    FROM Tiendas T
    WHERE NOT EXISTS (
        SELECT *
        FROM Frecuenta FREC
        WHERE FREC.idClte = CLTE.idClte
            AND FREC.idTienda = T. idTienda
    )
);
```

3

2

1

Seleccionamos todas las tiendas que NO tienen a dicha persona como cliente frecuente

CORRELACIÓN 2

Vemos si un determinado cliente frecuenta una determinada tienda

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

**MF 22.** *Mostrar la compañía con la que se realizaron más llamadas durante el día 16/10/06.*

COMPAÑIA.NOMBRE

MAX

LLAMADA.FECHA_HORA

**COMPAÑIA** (cif,nombre,tipo)
CP: cif
Único: nombre
VNN: nombre

JOIN1

JOIN2

**TARIFA** (tarifa,compañia,descripcion,coste)
CP: (tarifa,compañia)
CAj: compañia → COMPAÑIA (cif)
VNN: coste

2

1

**LLAMADA** (tf_origen,tf_destino,fecha_hora, duracion)
CP: (tf_origen,fecha_hora)
CAj: tf_origen → TELEFONO(numero)
CAj: tf_destino→ TELEFONO(numero)
VNN: duracion
VNN: tf_destino,duracion
Unico: (tf_destino, fecha_hora)

**TELEFONO** (numero,f_contrato,tipo,puntos, compañía,tarifa,cliente)
CP: numero
CAj: (tarifa,compañía) → TARIFA
CAj: compañía → COMPAÑIA (cif)
CAj: cliente → CLIENTE (dni)
Único: compañía, tarifa
VNN: compañía, tarifa,cliente

**CLIENTE**(dni,nombre,f_nac,direccion,cp,ciudad provincia)
CP: dni
VNN: nombre

BD MOVIFACTURA

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

***MF 22.*** ***Mostrar la compañía con la que se realizaron <u>más llamadas</u> durante el día 16/10/06***

COMPAÑIA.NOMBRE

MAX

LLAMADA.FECHA_HORA

Para cada *teléfono que interviene en una llamada*, necesitaremos obtener el nombre de su compañía. Por lo que, basándonos en las claves primarias-foráneas LLAMADA.tel_origen→TELEFONO.numero,TELEFONO.compañia→COMPAÑIA.cif, haremos sendos INNER JOIN para relacionar dichas tablas y filtrar las tuplas del 16/10/06:

Rabida

Hoja de Trabajo | Generador de Consultas

```
1 SELECT DISTINCT CIA.nombre
2 FROM (MF.Llamada LLAM inner join MF.Telefono TEL on LLAM.tf_origen=TEL.numero)
3       inner join MF.compañia CIA on TEL.compañia=CIA.cif
4       WHERE to_char(LLAM.fecha_hora, 'dd/mm/yy') = '16/10/06';
```

Resultado de la Consulta | SQL | Todas las Filas

| | NOMBRE |
|---|---|
| 1 | Kietostar |

Hacemos el INNER JOIN de las tablas que participan en la consulta, seleccionando el nombre de las empresas desde cuyos teléfonos se relizaron llamadas el 16/10/06

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

**MF 22.** *Mostrar la compañía con la que se realizaron más llamadas durante el día 16/10/06*

COMPAÑÍA.NOMBRE

MAX

LLAMADA.FECHA_HORA

A continuación, tendremos que agrupar por COMPAÑÍA para poder contabilizar el número de llamadas realizadas desde cada una de ellas (COUNT(*)), valor que utilizaremos para seleccionar aquélla desde la que se realizó un mayor número de llamadas.

Sin Título6.sql

Hoja de Trabajo de SQL

Hoja de Trabajo

Contabilizamos el número de llamadas realizadas desde los teléfonos de cada una de las empresas en dicha fecha

Consulta intermedia

```
1 SELECT CIA.nombre, COUNT(*) Num_Llamadas
2 FROM (MF.Llamada LLAM INNER JOIN MF.Telefono TEL ON LLAM.tf_origen=TEL.numero)
3       INNER JOIN MF.Compañia CIA on TEL.compañia=CIA.cif
4 WHERE to_char(LLAM.fecha_hora,'dd/mm/yy')='16/10/06'
5 GROUP BY CIA.nombre;
6
```

Hacemos la agrupación por nombre de compañía (⚠la selección no es una constante ni una función) y contabilizamos las tuplas

Resultado

| | NOMBRE | NUM_LLAMADAS |
|---|---|---|
| 1 | Kietostar | 3 |

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

***MF 22.*** *Mostrar la compañía con la que se realizaron más llamadas durante el día 16/10/06*

LLAMADA.FECHA_HORA → COMPAÑÍA.NOMBRE → MAX

**Determinamos cuál fue mayor número de llamadas realizadas desde una compañía durante el día 16/10/2006, para lo que haríamos la siguiente consulta:**

2

Consulta intermedia

Rabida

Hoja de Trabajo | Generador de Consultas

```
1  SELECT MAX(COUNT(*))
2  FROM MF.Llamada LLAM inner join MF.Telefono TEL on LLAM.tf_origen=TEL.numero
3  WHERE to_char(LLAM.fecha_hora, 'dd/mm/yy') = '16/10/06'
4  GROUP BY TEL.compañia;
```

Para poder relacionar las llamadas con las compañías, necesitamos hacer un INNER JOIN entre las tablas LLAMADA y TELEFONO, pues ésta última contiene el id de la compañía a la que pertenece el teléfono

Agrupamos por compañía

Resultado de la Cons | SQL | adas: 1 en 0,032 segundos

| | MAX(COUNT(*)) |
|---|---|
| 1 | 3 |

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

***MF 22.*** *Mostrar la compañía con la que se realizaron más llamadas durante el día 16/10/06*

COMPAÑIA.NOMBRE

MAX

LLAMADA.FECHA_HORA

Por último, para determinar la compañía desde la que se realizó un mayor número de llamadas, el día 16/10/2006, seleccionaremos, entre las compañías, aquélla cuyo número de llamadas, en dicho día, sea igual al máximo número de llamadas realizadas:

3

Rabida

Hoja de Trabajo | Generador de Consultas

```
1  SELECT DISTINCT CIA.nombre
2  FROM (MF.Llamada LLAM inner join MF.Telefono TEL on LLAM.tf_origen=TEL.numero)
3        inner join MF.compañia CIA on TEL.compañia=CIA.cif
4        WHERE to_char(LLAM.fecha_hora, 'dd/mm/yy') = '16/10/06'
5  GROUP BY CIA.nombre
6  HAVING COUNT(*) =( SELECT MAX(COUNT(*))
                      FROM MF.Llamada LLAM inner join MF.Telefono TEL on LLAM.tf_origen=TEL.numero
                      WHERE to_char(LLAM.fecha_hora, 'dd/mm/yy') = '16/10/06'
                      GROUP BY TEL.compañia );
```

Hacemos el INNER JOIN de las tablas que participan en la consulta

Contabiliza el número de llamadas realizadas desde teléfonos de las compañías

Seleccionamos la compañía desde la que se haya realizado un mayor número de llamadas el 16/10/06

Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 1 en 0,033 segun

| | NOMBRE |
|---|---|
| 1 | Kietostar |

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

TELEFONO.NUMERO

***MF 23.*** *Obtener los números de teléfono y los nombres de los abonados que han llamado a todos los números a los que se llamó desde el 654345345 en octubre de 2006*

CLIENTE.NOMBRE LLAMADA.TF_ORIGEN LLAMADA.FECHA_HORA DIVISIÓN

**COMPAÑIA** (cif, nombre, web)
- CP: cif
- Único: nombre
- VNN: nombre

**TARIFA** (tarifa,compañia,descripcion,coste)
- CP: (tarifa,compañia)
- CAj: compañia → COMPAÑIA (cif)
- VNN: coste

2

**LLAMADA** (tf_origen,tf_destino,fecha_hora, duracion)
- CP: (tf_origen,fecha_hora)
- CAj: tf_origen → TELEFONO(numero)
- CAj: tf_destino→ TELEFONO(numero)
- VNN: duracion
- VNN: tf_destino,duracion
- Unico: (tf_destino, fecha_hora)

1

**TELEFONO** (numero,contrato,tipo,puntos, compañía,tarifa,cliente)
- CP: numero
- CAj: (tarifa,compañía) → TARIFA
- CAj: compañía → COMPAÑIA (cif)
- CAj: cliente → CLIENTE (dni)
- Único: compañía, tarifa
- VNN: compañía, tarifa,cliente

JOIN

**CLIENTE** (dni,nombre,f_nac,direccion,cp,ciudad provincia)
- CP: dni
- VNN: nombre

BD MOVIFACTURA

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

TELEFONO.NUMERO

***MF 23.*** *Obtener los números de teléfono y los nombres de los abonados que han llamado a todos los números a los que se llamó desde el 654345345 en octubre de 2006*

CLIENTE.NOMBRE LLAMADA.TF_ORIGEN LLAMADA.FECHA_HORA DIVISIÓN

Como queremos obtener las tuplas de una tabla que se relacionan con TODAS las tuplas de otra, para ello, aplicaremos la operación DIVISIÓN, basada en el doble bloque NOT EXISTS.

Definimos el doble bloque EXISTS basándonos en la siguiente reformulación de la consulta: Para todos los teléfonos considerados, *no existe ningún teléfono al que se haya llamado desde el 654345345, en octubre de 2006, al que no se exista una llamada realizada desde el teléfono considerado*.

La estructura general de la consulta sería la siguiente:

1. **Seleccionar los números de teléfono y los nombres de los abonados**
   (INNER JOIN **de las tablas** CLIENTE **y** TELEFONO)
2. **NOT EXISTS**
   Seleccionar los teléfonos que recibieron llamadas del 654345345, en octubre de 2006
   ( LLAMADA: **tf_origen** ='654345345', **tf_destino** y **fecha_hora** '10/2006' )
3. **NOT EXISTS**
   Seleccionar todas las llamadas realizadas desde los teléfonos considerados y establecer la ⚠ ***correlación*** entre subconsultas

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

TELEFONO.NUMERO

***MF 23.*** *Obtener los números de teléfono y los nombres de los abonados que han llamado a todos los números a los que se llamó desde el 654345345 en octubre de 2006*

CLIENTE.NOMBRE LLAMADA.TF_ORIGEN LLAMADA.FECHA_HORA DIVISIÓN

- En primer lugar, como para los *teléfonos que realizan las llamadas* necesitamos obtener el nombre de su abonado, será necesario hacer un INNER JOIN con el que relacionaremos las tablas CLIENTE y TELEFONO.

1

Rabida

Consulta intermedia

Hoja de Trabajo | Generador de Consultas

```
1 SELECT TEL.numero, CIA.nombre
2 FROM MF.CLIENTE CIA INNER JOIN MF.TELEFONO TEL ON CIA.dni=TEL.cliente;
```

Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 10 en 0,111 segundos

| | NUMERO | NOMBRE |
|---|---|---|
| 1 | 654012012 | Jesús Ríos Lorca |
| 2 | 654234234 | Antonio Hierro Santander |
| 3 | 654123321 | Ramón Martínez Sabina |

...

## MF 23. Obtener los números de teléfono y los nombres de los abonados que han llamado a todos los números a los que se llamó desde el 654345345 en octubre de 2006

- Definimos el doble bloque EXISTS basándonos en la siguiente premisa: *no existe un teléfono al que se haya llamado desde el* 654345345 *en* octubre de 2006, *al que no se haya llamado desde el teléfono considerado.*

$\forall x: P(x) \leftrightarrow \neg\exists x: \neg P(x)$

Rabida

Consulta Find

Hoja de Trabajo | Generador de Consultas

```
1 SELECT TEL.numero, CIA.nombre
2 FROM MF.CLIENTE CIA INNER JOIN MF.TELEFONO TEL ON CIA.dni=TEL.cliente
3 WHERE NOT EXISTS (SELECT * FROM MF.LLAMADA LLAM1
4                   WHERE LLAM1.tf_origen='654345345'
5                   AND to_char(LLAM1.fecha_hora,'mm/yyyy') = '10/2006'
6                   AND NOT EXISTS (SELECT *
7                                   FROM MF.LLAMADA LLAM2
8                                   WHERE TEL.numero = LLAM2.tf_origen AND LLAM2.tf_destino=LLAM1.tf_destino
9                                     AND LLAM2.tf_origen<> '654345345'));
```

1 2 3

Tenemos que excluir las llamadas con tf_origen 654345345 para que no aparezca en la solución

Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 1 en 0,032 segund

| NUMERO | NOMBRE |
|---|---|
| 678234234 | Rita Martínez de Castro |

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN5

SUM

**MF 24.** ***Mostrar el nombre de cada cliente junto con coste total de las llamadas que realiza con cada compañía. El resultado debe mostrarse ordenado descendentemente por cliente y ascendentemente por compañía***

**COMPAÑIA** (cif,nombre,web)
CP: cif
Único: nombre
VNN: nombre

**TARIFA** (tarifa,compañia,descripcion,coste)
CP: (tarifa,compañia)
CAj: compañia → COMPAÑIA (cif)
VNN: coste

**LLAMADA** (tf_origen,tf_destino,fecha_hora, duracion)
CP: (tf_origen,fecha_hora)
CAj: tf_origen → TELEFONO(numero)
CAj: tf_destino→ TELEFONO(numero)
VNN: duracion
VNN: tf_destino,duracion
Unico: (tf_destino, fecha_hora)

**TELEFONO** (número,f_contrato,tipo,puntos, compañía,tarifa,cliente)
CP: numero
CAj: (tarifa,compañía) → TARIFA
CAj: compañía → COMPAÑIA (cif)
CAj: cliente → CLIENTE (dni)
Único: compañía, tarifa
VNN: compañía, tarifa,cliente

**CLIENTE** (nombre,f_nac,direccion,cp,ciudad provincia)
CP: dni
VNN: nombre

JOIN1 JOIN2 JOIN3 JOIN4

BD MOVIFACTURA

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

SUM

**MF 24.** ***Mostrar el nombre de cada cliente junto con coste total de las llamadas que realiza con cada compañía. El resultado debe mostrarse ordenado descendentemente por cliente y ascendentemente por compañía***

**Observando la estructura de la BD y los datos que hemos de manejar en la consulta, se deduce que será necesario realizar un** INNER JOIN **de** ⚠TODAS LAS TABLAS:

Rabida

Hoja de Trabajo | Generador de Consultas

```
1  SELECT *
2  FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
3        INNER JOIN MF.Tarifa   TAR ON TAR.tarifa=TEL.tarifa AND TAR.compañia=TEL.compañia
4        INNER JOIN MF.Compañia CIA ON CIA.cif=TAR.compañia
5        INNER JOIN MF.Llamada LLAM ON LLAM.tf_origen=TEL.numero;
6
```

Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 30 en 0,052 segundos

| | DNI | NOMBRE | F_NAC | DIRECCION | CP | CIUDAD |
|---|---|---|---|---|---|---|
| 1 | 35000001P | Ramón Martínez Sabina | 12/02/69 | C/ Melancolía nº 7 | 23400 | Úbeda |
| 2 | 35000001P | Ramón Martínez Sabina | 12/02/69 | C/ Melancolía nº 7 | 23400 | Úbeda |
| 3 | 35000001P | Ramón Martínez Sabina | 12/02/69 | C/ Melancolía nº 7 | 23400 | Úbeda |

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

SUM

***MF 24.*** ***Mostrar el nombre de cada cliente junto con coste total de las llamadas que realiza con cada compañía. El resultado debe mostrarse ordenado descendentemente por cliente y ascendentemente por compañía***

**Una vez realizado el** INNER JOIN **de las tablas tendremos que tener en cuenta lo siguiente:**

- **Para calcular el** coste **de una llamada, lo haremos en base a su** duración, **que está expresada en *segundos*, y al coste de la tarifa contratada** (precio por minuto**):**

  **(LLAM.**duracion***TAR.**coste**) /60**

- **Para calcular el** coste total **de las llamadas por compañía, utilizaremos la función** SUM **aplicada a la expresión del cálculo del *coste de cada llamada:***

  **SUM((LLAM.**duracion***TAR.**coste**) /60)**

- **Para poder mostrar el nombre de cada cliente junto a los costes totales de las llamadas realizadas con cada compañía, será necesario** agrupar **las tuplas resultantes por** cliente **y por** compañía**.**

- **Por último, aplicaremos un doble criterio de** ordenación **para las tuplas resultantes: *descendente* por *cliente* y *ascendente* para *compañía.***

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

**MF 24.** *Mostrar el nombre de cada cliente junto con coste total de las llamadas que realiza con cada compañía. El resultado debe mostrarse ordenado descendentemente por cliente y ascendentemente por compañía*

SUM

Rabida

CALCULAMOS LOS COSTES TOTALES

Hoja de Trabajo | Generador de Consultas

```
1 SELECT CLTE.nombre AS Cliente ,CIA.nombre AS Compañia, SUM(LLAM.duracion/60*TAR.coste) AS CosteTOTAL
2 FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
3       INNER JOIN MF.Tarifa   TAR ON TAR.tarifa=TEL.tarifa AND TAR.compañia=TEL.compañia
4       INNER JOIN MF.Compañia CIA ON CIA.cif=TAR.compañia
5       INNER JOIN MF.Llamada LLAM ON LLAM.tf_origen=TEL.numero
6 GROUP BY CLTE.nombre,CIA.nombre
7 ORDER BY Cliente DESC, Compañia ASC
8
```

AGRUPAMOS y ORDENAMOS

Hacemos el INNER JOIN de todas las tablas

Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 8 en 0,048 segundos

| | CLIENTE | COMPAÑIA | COSTETOTAL |
|---|---|---|---|
| 1 | Rita Martínez de Castro | Aotra | 11,964 |
| 2 | Rita Martínez de Castro | Kietostar | 4,113333333333333333333333333333333334 |
| 3 | Ricardo Reyes Neruda | Aotra | 4,0225 |
| 4 | Ramón Martínez Sabina | Kietostar | 10,180000000000000000000000000000000000 |
| 5 | Ramón Martínez Sabina | Petafón | 2,435166666666666666666666666666666666 |
| 6 | María Machado Ruiz | Petafón | 2,142833333333333333333333333333333333 |
| 7 | José García Márquez | Kietostar | 4,625 |
| 8 | Jesús Ríos Lorca | Kietostar | 3,775 |

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

**MF 25.** *Para cada cliente residente en la provincia de 'La Coruña', mostrar la duración de todas las llamadas realizadas a clientes residentes en 'Jaén'*

GROUP IN CLIENTE.PROVINCIA SUM IN CLIENTE.PROVINCIA

**COMPAÑIA** (cif, nombre, web)
CP: cif
Único: nombre
VNN: nombre

**TARIFA** (tarifa,compañia,descripcion,coste)
CP: (tarifa,compañia)
CAj: compañia → COMPAÑIA (cif)
VNN: coste

JOIN 2 — 2

**LLAMADA** (tf_origen,tf_destino,fecha_hora, duracion)
CP: (tf_origen,fecha_hora)
CAj: tf_origen → TELEFONO(numero)
CAj: tf_destino→ TELEFONO(numero)
VNN: duracion
VNN: tf_destino,duracion
Unico: (tf_destino, fecha_hora)

**TELEFONO** (número,f_contrato,tipo,puntos, compañía,tarifa,cliente)
CP: numero
CAj: (tarifa,compañía) → TARIFA
CAj: compañía → COMPAÑIA (cif)
CAj: cliente → CLIENTE (dni)
Único: compañía, tarifa
VNN: compañía, tarifa,cliente

1 — JOIN 1

**CLIENTE**(dni, nombre,f_nac,direccion,cp,ciudad provincia)
CP: dni
VNN: nombre

BD MOVIFACTURA

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

***MF 25.*** *Para cada cliente residente en la provincia de 'La Coruña', mostrar la duración de todas las llamadas realizadas a clientes residentes en 'Jaén'*

GROUP IN CLIENTE.PROVINCIA SUM IN CLIENTE.PROVINCIA

- Observando la estructura de la BD y los datos que hemos de manejar en la consulta, se deduce que será necesario realizar un INNER JOIN de las tablas CLIENTE, TELÉFONO y LLAMADA.
- Obtendremos, mediante un INNER JOIN de las tablas CLIENTES y TELÉFONOS, los conjuntos de teléfonos pertenecientes a clientes de las provincias de La Coruña y de Jaén.
- Mediante la tabla LLAMADAS, seleccionamos aquellas llamadas cuyo *teléfono origen* pertenece al conjunto de teléfonos de clientes de *La Coruña*, y cuyo *teléfono destino* pertenezca al conjunto de teléfonos de clientes de *Jaén*.
- Por último, agruparemos los datos por nombre de cliente.

1

Rabida1.sql

Hoja de Trabajo de SQL | Historial

Hoja de Trabajo | Generador de Consultas

```
1  SELECT *
2  FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
3        INNER JOIN MF.Llamada LLAM ON LLAM.tf_origen=TEL.numero;
```

Hacemos el INNER JOIN de las tablas CLIENTE, TELÉFONO y LLAMADA

# 6. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

**MF 25.** *Para cada cliente residente en la provincia de 'La Coruña', mostrar la duración de todas las llamadas realizadas a clientes residentes en 'Jaén'*

GROUP IN CLIENTE.PROVINCIA SUM IN CLIENTE.PROVINCIA

2

CALCULAMOS LOS COSTES TOTALES

Rabida1.sql

Hoja de Trabajo de SQL | Historial

Rabida

Hoja de Trabajo | Generador de Consultas

SUMA de las duraciones de las llamadas

```
1  SELECT CLTE.nombre Cliente, SUM(LLAM.duracion) Total
2  FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
3        INNER JOIN MF.Llamada LLAM ON LLAM.tf_origen=TEL.numero
4  WHERE LLAM.tf_origen IN ( SELECT TEL.numero
5                            FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
6                            WHERE CLTE.provincia = 'La Coruña'
7                          )
8    AND LLAM.tf_destino IN ( SELECT TEL.numero
9                            FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
10                           WHERE CLTE.provincia = 'Jaén'
11                         )
12 GROUP BY CLTE.nombre
13
```

Teléfonos pertenecientes a clientes de La Coruña

Teléfonos pertenecientes a clientes de Jaén

AGRUPAMOS por cliente

Salida de Script | Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 1 en 0,07 [illegible]dos

| | CLIENTE | TOTAL |
|---|---|---|
| 1 | Rita Martínez de Castro | 2753 |

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN5

**MF 26.** ***Mostrar los nombres de todos los clientes que hayan realizado más de 5 llamadas.***

HAVING

CLIENTE.NOMBRE

COUNT

**COMPAÑIA** (cif, nombre, web)
CP: cif
Único: nombre
VNN: nombre

JOIN2

2

**TARIFA** (tarifa,compañia,descripcion,coste)
CP: (tarifa,compañia)
CAj: compañia → COMPAÑIA (cif)
VNN: coste

**LLAMADA** (tf_origen,tf_destino,fecha_hora, duracion)
CP: (tf_origen,fecha_hora)
CAj: tf_origen → TELEFONO(numero)
CAj: tf_destino→ TELEFONO(numero)
VNN: duracion
VNN: tf_destino,duracion
Unico: (tf_destino, fecha_hora)

1

JOIN1

**TELEFONO** (número,f_contrato,tipo,puntos, compañía,tarifa,cliente)
CP: numero
CAj: (tarifa,compañía) → TARIFA
CAj: compañía → COMPAÑIA (cif)
CAj: cliente → CLIENTE (dni)
Único: compañía, tarifa
VNN: compañía, tarifa,cliente

**CLIENTE**(dni,nombre,fnac,direccion,cp,ciudad provincia)
CP: dni
VNN: nombre

BD MOVIFACTURA

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

***MF 26.*** ***Mostrar los nombres de todos los clientes que hayan realizado más de 5 llamadas.***

CLIENTE.NOMBRE

COUNT

HAVING

- **Observando la estructura de la BD y los datos que hemos de manejar en la consulta, se deduce que será necesario realizar un** INNER JOIN **de las tablas** CLIENTE, TELÉFONO **y** LLAMADA.
- **En el SELECT especificamos el** nombre del cliente **y la función de agrupación COUNT, mediante la que contabilizamos** el número de llamadas realizadas **por el cliente.**
- **Agrupamos los datos por** nombre de cliente, **estableciendo como condición de selección de grupo que el número de llamadas realizadas sea mayor que 5 (COUNT(*)>5).**

1

Rabida1.sql

Hoja de Trabajo de SQL | Historial

Rabida

Hoja de Trabajo | Generador de Consultas

```
1  SELECT *
2  FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
3        INNER JOIN MF.Llamada LLAM ON LLAM.tf_origen=TEL.numero;
```

Hacemos el INNER JOIN de las tablas CLIENTE, TELÉFONO y LLAMADA

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

**MF 26.** *Mostrar los nombres de todos los clientes que hayan realizado más de 5 llamadas.*

HAVING — CLIENTE.NOMBRE — COUNT

2

Sin Título6.sql

Hoja de Trabajo de SQL | Historial

ETSI

Hoja de Trabajo | Generado

Contabilizamos el número de llamadas realizadas por cada cliente

```
7  SELECT CLTE.nombre, COUNT(*) Num_Llamadas
8  FROM (MF.Cliente CLTE  INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
9       INNER JOIN MF.Llamada LLAM ON LLAM.tf_origen=TEL.numero
10 GROUP BY CLTE.nombre
```

AGRUPAMOS por cliente

Hacemos el INNER JOIN de las tablas que participan en la consulta

Resultado de la Consulta

SQL | Todas las Filas Recuperadas: 6 en 0,038 segundos

| | NOMBRE | NUM_LLAMADAS |
|---|---|---|
| 1 | Ramón Martínez Sabina | 8 |
| 2 | Rita Martínez de Castro | 9 |
| 3 | Ricardo Reyes Neruda | 4 |
| 4 | María Machado Ruiz | 3 |
| 5 | José García Márquez | 3 |
| 6 | Jesús Ríos Lorca | 3 |

Número de llamadas realizadas por cada cliente

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

**MF 26.** *Mostrar los nombres de todos los clientes que hayan realizado más de 5 llamadas.*

CLIENTE.NOMBRE

COUNT

HAVING

3

Rabida1.sql

Hoja de Trabajo de SQL | Historial

Rabida

Hoja de Trabajo | Generador

Contabilizamos el número de llamadas realizadas por cada cliente

```
1  SELECT CLTE.nombre, COUNT(*)
2  FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
3        INNER JOIN MF.Llamada LLAM ON LLAM.tf_origen=TEL.numero
4  GROUP BY CLTE.nombre
5  HAVING COUNT(*)>5;
```

AGRUPAMOS por cliente

Hacemos el INNER JOIN de las tablas que participan en la consulta

Seleccionamos sólo los clientes que hayan realizado más de 5 llamadas

Salid

SQL | Todas las Filas Recuperadas: 2 en 0,031 segundos

| | NOMBRE | COUNT(*) |
|---|---|---|
| 1 | Ramón Martínez Sabina | 8 |
| 2 | Rita Martínez de Castro | 9 |

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

***MF 27.*** ***Mostrar el nombre de aquellos clientes que hayan contratado con cualquier compañía, de media, una tarifa superior a la tarifa media total.***

CLIENTE.NOMBRE HAVING AVG

**COMPAÑIA** (cif, nombre, web)
- CP: cif
- Único: nombre
- VNN: nombre

**TARIFA** (tarifa,compañia,descripcion,coste)
- CP: (tarifa,compañia)
- CAj: compañia → COMPAÑIA (cif)
- VNN: coste

JOIN2 2

**LLAMADA** (tf_origen,tf_destino,fecha_hora, duracion)
- CP: (tf_origen,fecha_hora)
- CAj: tf_origen → TELEFONO(numero)
- CAj: tf_destino→ TELEFONO(numero)
- VNN: duracion
- VNN: tf_destino,duracion
- Unico: (tf_destino, fecha_hora)

1 JOIN1

**TELEFONO** (numero,f_contrato,tipo,puntos, compañía,tarifa,cliente)
- CP: numero
- CAj: (tarifa,compañía) → TARIFA
- CAj: compañía → COMPAÑIA (cif)
- CAj: cliente → CLIENTE (dni)
- Único: compañía, tarifa
- VNN: compañía, tarifa,cliente

**CLIENTE**(dni,nombre,fac,direccion,cp,ciudad provincia)
- CP: dni
- VNN: nombre

BD MOVIFACTURA

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

***MF 27.*** ***Mostrar el nombre de aquellos clientes que hayan contratado con cualquier compañía, de media, una tarifa superior a la tarifa media total.***

CLIENTE.NOMBRE HAVING AVG

- **Observando la estructura de la BD y los datos que hemos de manejar en la consulta, se deduce que será necesario realizar un** INNER JOIN **de las tablas** CLIENTE, TELÉFONO **y** TARIFA.
- **En el SELECT especificamos el** nombre del cliente **y la función de agrupación AVG, con la que obtenemos** el valor medio de las tarifas contratadas **por el cliente.**
- **Agrupamos los datos por** nombre de cliente, **estableciendo como condición de selección que** el valor medio de las tarifas contratadas **por el cliente sea mayor que** el valor medio total de las tarifas.

1

Rabida1.sql

Hoja de Trabajo de SQL | Historial

Hoja de Trabajo | Generador de Consultas

Hacemos el INNER JOIN de las tablas CLIENTE, TELÉFONO y TARIFA

```
SELECT *
FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
      INNER JOIN MF.Tarifa TAR USING(tarifa, compañia);
```

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

**MF 27.** *Mostrar el nombre de aquellos clientes que hayan contratado con cualquier compañía, de media, una tarifa superior a la tarifa media total.*

CLIENTE.NOMBRE — HAVING — AVG

2

Rabida1.sql

Hoja de Trabajo de SQL | Historial

Hoja de Trabajo | Generador de Consultas

Rabida

```
1 SELECT CLTE.nombre Cliente, AVG(TAR.coste) Media
2 FROM (MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
3       INNER JOIN MF.Tarifa TAR USING(tarifa, compañía)
4 GROUP BY CLTE.nombre
5 HAVING AVG(TAR.coste) >=(SELECT AVG(TAR.coste) FROM MF.Tarifa TAR);
```

Calculamos el coste medio de las tarifas contratadas por el cliente

AGRUPAMOS por cliente

Hacemos el INNER JOIN de las tablas que participan en la consulta

Seleccionamos sólo los clientes cuya tarifa media sea superior a la tarifa media global

Calculamos la tarifa media global

n 0,03 segundos

| | CLIENTE | MEDIA |
|---|---|---|
| 1 | Antonio Hierro Santander | 0,25 |
| 2 | Ramón Martínez Sabina | 0,195 |
| 3 | María Machado Ruiz | 0,23 |
| 4 | Jesús Ríos Lorca | 0,25 |

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN5

**MF 28.** ***Mostrar los nombres de los clientes que hayan realizado llamadas a teléfonos de la compañía 'Kietostar' por un coste total menor de 100€.***

CLIENTE.NOMBRE — IN — COMPAÑIA.NOMBRE — HAVING

**COMPAÑIA** (cif, nombre, web)
- CP: cif
- Único: nombre
- VNN: nombre

**TARIFA** (tarifa,compañia,descripcion,coste)
- CP: (tarifa,compañia)
- CAj: compañia → COMPAÑIA (cif)
- VNN: coste

**LLAMADA** (tf_origen,tf_destino,fecha_hora duracion)
- CP: (tf_origen,fecha_hora)
- CAj: tf_origen → TELEFONO(numero)
- CAj: tf_destino→ TELEFONO(numero)
- VNN: duracion
- VNN: tf_destino,duracion
- Unico: (tf_destino, fecha_hora)

**TELEFONO** (numero,f_contrato,tipo,puntos, compañía,tarifa,cliente)
- CP: numero
- CAj: (tarifa,compañía) → TARIFA
- CAj: compañía → COMPAÑIA (cif)
- CAj: cliente → CLIENTE (dni)
- Único: compañía, tarifa
- VNN: compañía, tarifa,cliente

**CLIENTE**(dni,nombre,fnac,direccion,cp,ciudad provincia)
- CP: dni
- VNN: nombre

JOIN3 — JOIN2 — JOIN1

BD MOVIFACTURA

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

***MF 28.*** ***Mostrar los nombres de los clientes que hayan realizado llamadas a teléfonos de la compañía 'Kietostar' por un coste menor de 100€.***

CLIENTE.NOMBRE IN COMPAÑIA.NOMBRE SUM HAVING

- **Observando la estructura de la BD y los datos que hemos de manejar en la consulta, se deduce que será necesario realizar un** INNER JOIN **de las tablas** CLIENTE, TELÉFONO, LLAMADA **y** TARIFA.
- **En el SELECT especificamos el** nombre del cliente**, que será el valor por el que agruparemos.**
- **Tendremos que obtener el conjunto de teléfonos pertenecientes a la compañía** KietoStar**, para lo cual será necesario realizar un** INNER JOIN **entre las tablas** TELÉFONO **y** COMPAÑÍA**. Esta información nos permitirá determinar si** el teléfono destino de una llamada pertenece o no a la compañía KietoStar.
- **Agrupamos los datos por** nombre de cliente**, estableciendo como condición de selección que** el coste total de las llamadas realizadas **por el cliente a teléfonos de la compañía KietoStar sea inferior a 100€.**

1

Rabida1.sql

Hoja de Trabajo de SQL | Historial

Hacemos el INNER JOIN de las tablas CLIENTE, TELÉFONO, LLAMADA y TARIFA

Hoja de Trabajo | Generador de Consultas

```
1 SELECT *
2 FROM ((MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
3       INNER JOIN MF.Tarifa TAR USING(tarifa, compañia))
4       INNER JOIN MF.Llamada LLAM ON LLAM.tf_origen=TEL.numero;
```

# 5. LENGUAJE DE CONSULTA (LMD) –SESIÓN 5

**MF 28.** *Mostrar los nombres de los clientes que hayan realizado llamadas a teléfonos de la compañía 'Kietostar' por un coste menor de 100€.*

CLIENTE.NOMBRE — IN — COMPAÑIA.NOMBRE — HAVING

2

Rabida1.sql

Hoja de Trabajo de SQL | Historial

Hoja de Trabajo | Generador de Consultas

```
1  SELECT CLTE.nombre Cliente
2  FROM ((MF.Cliente CLTE INNER JOIN MF.Telefono TEL ON TEL.cliente=CLTE.dni)
3        INNER JOIN MF.Tarifa TAR USING(tarifa, compañia))
4        INNER JOIN MF.Llamada LLAM ON LLAM.tf_origen=TEL.numero
5  WHERE LLAM.tf_destino IN ( SELECT TEL.numero
6                             FROM MF.Telefono TEL INNER JOIN MF.Compañia CIA ON TEL.compañia = CIA.cif
7                             WHERE CIA.nombre='Kietostar')
8  GROUP BY CLTE.nombre
9  HAVING SUM((LLAM.duracion*TAR.coste)/60)<100;
```

Hacemos el INNER JOIN de las tablas que participan en la consulta

AGRUPAMOS por cliente

Obtenemos los teléfonos pertenecientes a la compañía 'KietoStar'

Seleccionamos sólo los clientes cuyo suma total de importes sea inferior a 100€

| | CLIENTE |
|---|---|
| 1 | Ramón Martínez Sabina |
| 2 | Ricardo Reyes Neruda |
| 3 | Rita Martínez de Castro |
| 4 | José García Márquez |